PROJECT DOCUMENTATION

(Leave Management System)

---

1. Introduction

The Leave Management System is a web-based application developed to replace manual leave request processes in organizations. The system allows employees to submit leave requests and enables managers to review, approve, or reject those requests efficiently.

The application is built using:

- **Frontend:** React + Tailwind CSS
- **Backend:** Golang (Gin framework)
- **Database:** PostgreSQL

---

2. Problem Statement

Currently, leave management is handled manually, which leads to:

- delays in approval
- lack of transparency
- difficulty in tracking leave status

This system solves these issues by providing a **digital, role-based solution**.

---

3. System Overview

The system follows a **3-tier architecture**:

Frontend (React)
↓
Backend API (Golang - Gin)
↓
Database (PostgreSQL)

4. User Roles

The system uses **Role-Based Access Control (RBAC)**.

Employee

Registers using **"Sign Up as Employee"**
Can:

- Request leave
- View own leave requests
- Track leave status

Manager (Admin role)

Registers using **"Sign Up as Manager"**
Can:

- View all leave requests
- Approve or reject leave requests
- Manage employees

---

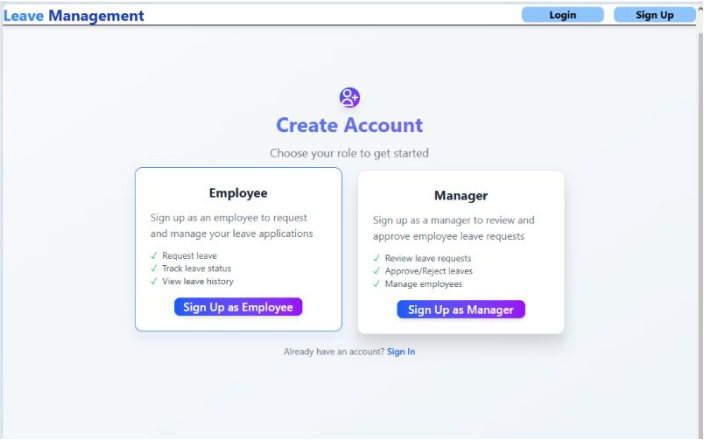
5. System Workflow

Step 1: User Registration (Sign Up)

Users can create an account by selecting their role:

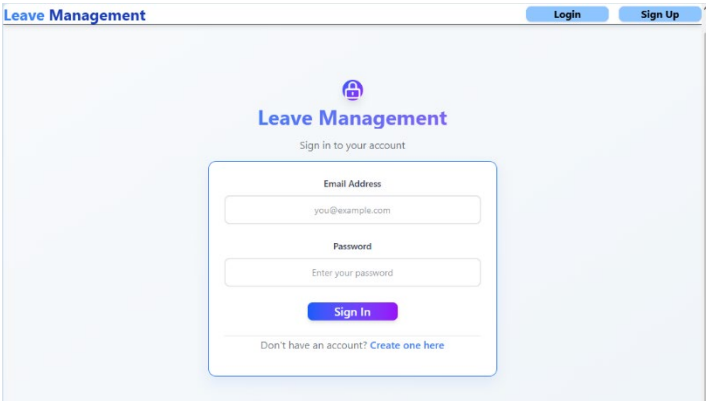
- Sign Up as Employee
- Sign Up as Manager

This determines system permissions.



Step 2: Login

- User logs in using:
  - email
  - password
- System returns:
  - JWT token
  - user role

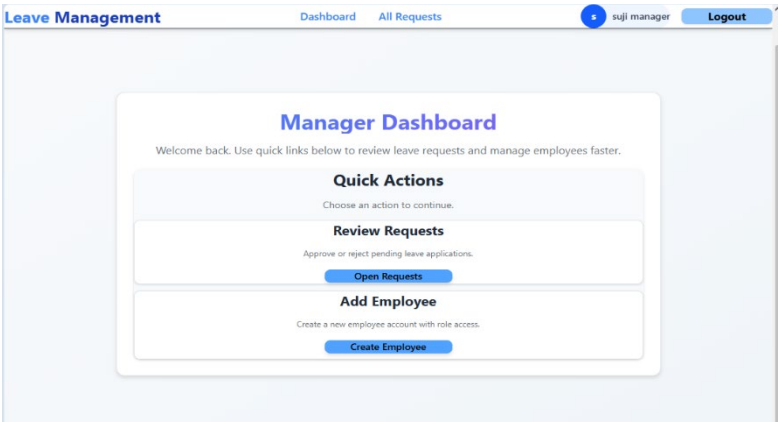


Step 3: Role-Based Redirection

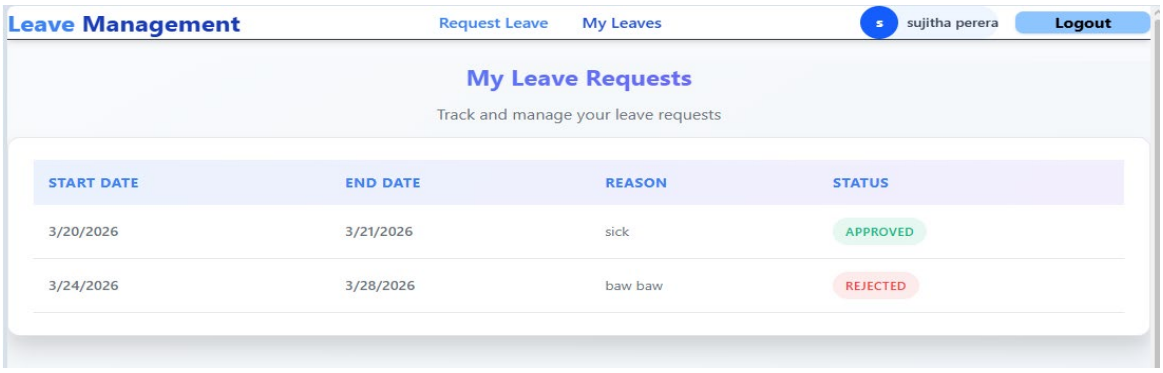
After login:

1. Employee → My Leaves page
2. Manager → Dashboard

2.



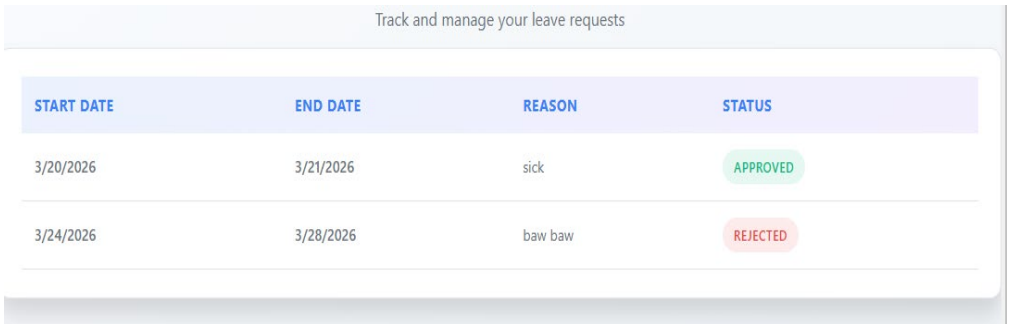
1.



(Employee Section)

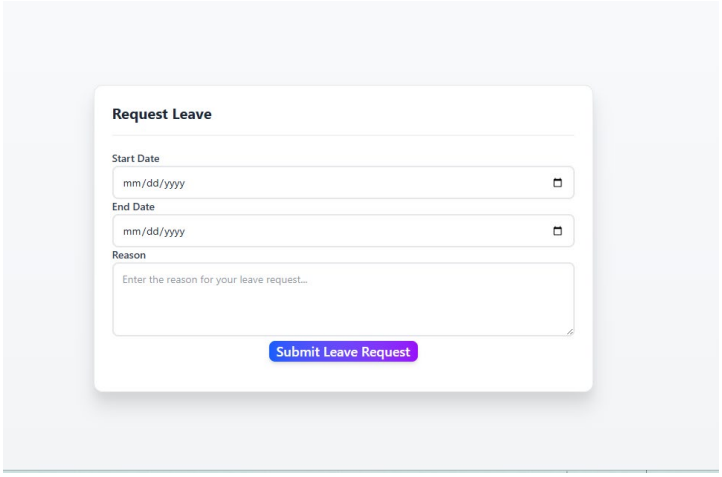
Step 4: Employee Views Own Leaves

- Employee can see:
  - leave details
  - status (PENDING / APPROVED / REJECTED)



Step 5: Employee Requests Leave

- Employee fills:
  - start date
  - end date
  - reason
- Leave saved as PENDING



(Manager Section)

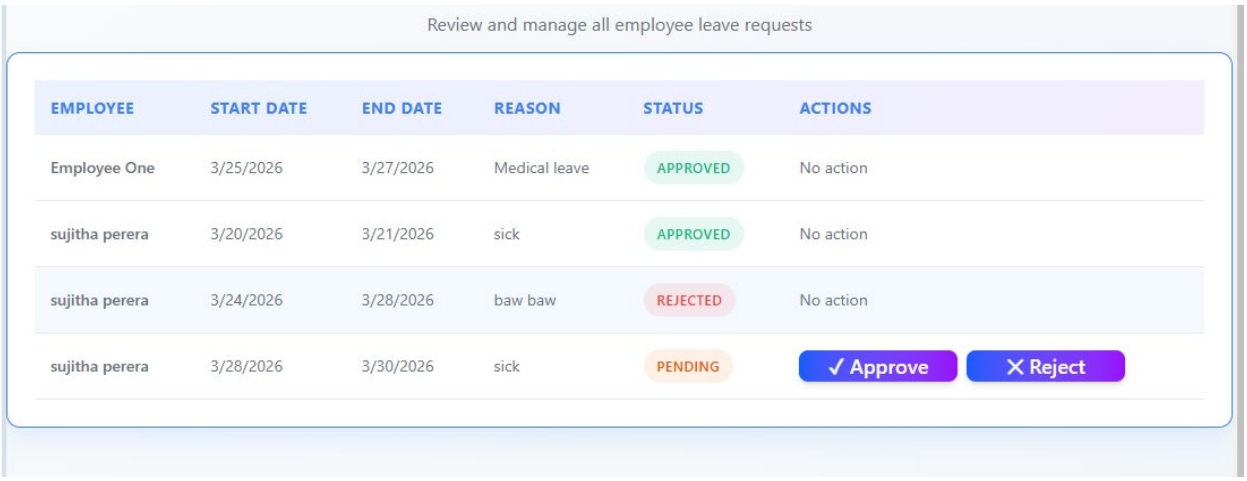
Step 6: Manager Views All Leaves

- Manager can see all employee requests

EMPLOYEE	START DATE	END DATE	REASON	STATUS	ACTIONS
Employee One	3/25/2026	3/27/2026	Medical leave	APPROVED	No action
sujitha perera	3/20/2026	3/21/2026	sick	APPROVED	No action
sujitha perera	3/24/2026	3/28/2026	baw baw	REJECTED	No action

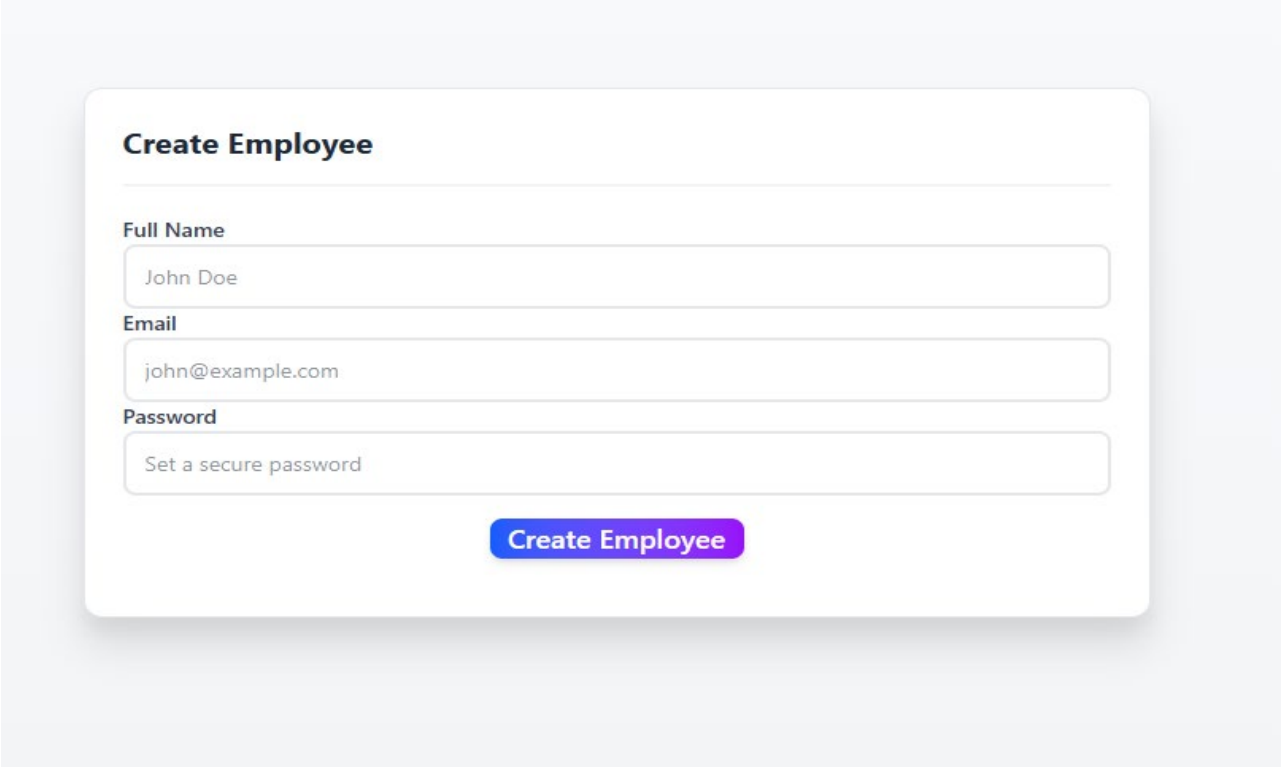
Step 6: Manager Approves / Rejects

- Manager clicks:
  - Approve → status = APPROVED
  - Reject → status = REJECTED

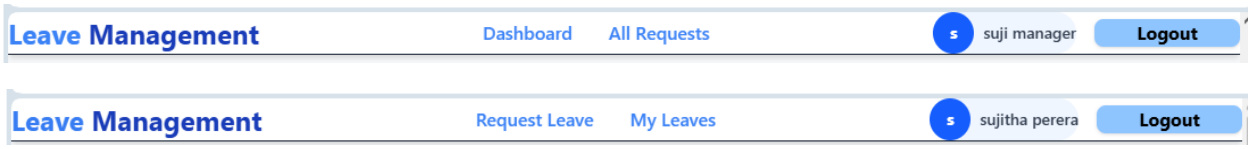


EMPLOYEE	START DATE	END DATE	REASON	STATUS	ACTIONS
Employee One	3/25/2026	3/27/2026	Medical leave	APPROVED	No action
sujitha perera	3/20/2026	3/21/2026	sick	APPROVED	No action
sujitha perera	3/24/2026	3/28/2026	baw baw	REJECTED	No action
sujitha perera	3/28/2026	3/30/2026	sick	PENDING	✓ Approve ✕ Reject

Step 7: Manager can create employee



Step 8: Both Manager and Employee can logout



6 . DATABASE DESIGN

Project consist **2 main tables**:

1. users
2. leaves

USERS TABLE: Store all users (Employee + Manager)

Structure:

```
CREATE TABLE users (
   id SERIAL PRIMARY KEY,
   full_name VARCHAR(100) NOT NULL,
   email VARCHAR(120) UNIQUE NOT NULL,
   password_hash TEXT NOT NULL,
   role VARCHAR(20) NOT NULL CHECK (role IN (EMPLOYEE , MANAGER )),
   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

LEAVES TABLE : Store all leave requests

Structure:

```
CREATE TABLE leave_requests (
   id SERIAL PRIMARY KEY,
   user_id INT NOT NULL,
   start_date DATE NOT NULL,
   end_date DATE NOT NULL,
   reason TEXT NOT NULL,
   status VARCHAR(20) DEFAULT PENDING
      CHECK (status IN (PENDING , APPROVED , REJECTED )),
   reviewed_by INT,
   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

   FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
   FOREIGN KEY (reviewed_by) REFERENCES users(id)
);
```

RELATIONSHIP

User (1) - (Many) Leave Requests

One user can have many leave requests.

7 . API Endpoints

Auth

- POST /register → register user
- POST /login → login

Users (Manager only)

- GET /users → get all users
- POST /users → create employee

Leave (Employee)

- POST /leave → request leave
- GET /leave/my → my leaves

Leave (Manager)

- GET /leave → all leaves
- PUT /leave/{id}/approve → approve
- PUT /leave/{id}/reject → reject

---

8 . Security Design

- Passwords are hashed using **bcrypt**
- Authentication handled using **JWT tokens**
- Role-based middleware restricts access:
    - Employee routes
    - Manager routes

---

9 . UI/UX DESIGN

The system includes a role selection sign-up interface, allowing users to clearly understand their permissions before creating an account.
Features:

- Two-card layout:
    - Employee
    - Manager
- Each card explains:
    - available actions
- Clear call-to-action buttons:
    - "Sign Up as Employee"
    - "Sign Up as Manager"